Informe de Producción
Shadow-Score Académico (v5)

Equipo de desarrollo

6 de mayo de 2026

Resumen

El proyecto Shadow-Score Académico consiste en una aplicación web interactiva que permite a estudiantes universitarios reflexionar sobre la distribución de su tiempo entre cargas domésticas, laborales y académicas, y visualizar posibles afectaciones en su rendimiento académico desde una perspectiva de género. Esta versión 5 incorpora un modelo matemático multifactorial calibrado con la ENUT 2024-2025 del DANE y un asistente de inteligencia artificial que genera planes de acción personalizados basados en el coste de oportunidad calculado. Adicionalmente, se ha desarrollado un **módulo administrativo** que permite a la universidad cargar archivos CSV con datos de múltiples estudiantes (un archivo por facultad), procesarlos mediante un flujo ETL implementado con estructuras internas de Python (Pandas) y presentar resultados agregados en dashboards interactivos, sin utilizar IA para preservar la confidencialidad de los datos sensibles. La solución se construye con Streamlit y se describe en detalle: arquitectura, modelo ajustado, integración de IA e interfaz.

1. Introducción

El proyecto Shadow-Score Académico consiste en una **aplicación web interactiva** que permite a estudiantes universitarios reflexionar sobre la distribución de su tiempo entre cargas domésticas, laborales y académicas, y visualizar posibles afectaciones en su rendimiento académico desde una perspectiva de género. La herramienta se construye con base en fundamentos investigativos previos y responde a la retroalimentación recibida, ampliando su enfoque hacia la autorreflexión y la acción, mediante la incorporación de un asistente de inteligencia artificial que genera recomendaciones personalizadas.

El presente informe describe la solución técnica desarrollada: su arquitectura, modelo matemático ajustado (v5), interfaz de usuario e integración del asistente de IA. Además, se incorpora un **módulo administrativo** destinado a directivos y personal de bienestar universitario, que permite el análisis masivo de datos a través de la carga de archivos CSV, procesamiento ETL (Extracción, Transformación y Carga) con estructuras internas de Python, y la generación de dashboards institucionales sin exposición de datos sensibles a servicios externos. El resultado final es una aplicación desplegada en Streamlit Cloud, accesible públicamente, que atiende tanto la necesidad de reflexión individual como la de toma de decisiones institucional.

2. Objetivos de producción

2.1. Objetivo general

Desarrollar una aplicación web interactiva que integre un modelo multifactorial de cálculo de fatiga y potencial afectación académica, basado en variables de carga doméstica, laboral, académica y contextuales, con un enfoque de autorreflexión y sensibilización sobre las brechas de género, y que ofrezca planes de acción personalizados mediante IA, además de un módulo administrativo para procesamiento masivo (CSV) y visualización agregada mediante dashboards.

2.2. Objetivos específicos

1. Implementar un modelo matemático que considere carga doméstica, carga laboral, composición del hogar, estrato, género y percepción subjetiva, para generar indicadores de fatiga, tiempo efectivo de estudio y un Shadow-Score cualitativo.

2. Diseñar una interfaz de usuario intuitiva para el rol estudiante que recoja las variables definidas y presente resultados mediante gráficos interactivos, métricas y elementos reflexivos.

3. Simular escenarios de corresponsabilidad y apoyos institucionales (reducción de carga doméstica, acceso a servicios de cuidado) que permitan comparar situaciones actuales y mejoradas.

4. Incluir información contextual (datos DANE, referencias a ODS) y recomendaciones de apoyo institucional.

5. Integrar un asistente basado en IA (vía API externa) que, utilizando el coste de oportunidad calculado, genere planes de acción personalizados para el usuario.

6. Diseñar un módulo administrativo que permita cargar archivos CSV con identificadores anónimos de estudiantes, procesarlos mediante un flujo ETL implementado con estructuras internas de Python (Pandas) y aplicar el mismo modelo matemático a cada registro.

7. Implementar dashboards de presentación de resultados agregados para el rol administrativo, que muestren métricas institucionales (distribución de Shadow-Score, brechas de género, etc.) sin utilizar IA debido a la sensibilidad de los datos.

8. Garantizar la escalabilidad, mantenibilidad y accesibilidad de la aplicación, utilizando buenas prácticas de desarrollo.

3. Arquitectura técnica

La aplicación se construye en **Python** utilizando **Streamlit** como framework de frontend.

3.1. Tecnologías y librerías

Tecnología	Uso
Python 3.10+	Lenguaje base
Streamlit	Interfaz de usuario, gestión de estado, despliegue
Pandas	Manejo de datos (almacenamiento temporal y procesamiento ETL
NumPy	Operaciones matemáticas y vectorización
Plotly	Gráficos interactivos (barras, radar, medidores)
NVIDIA API (Mistral Medium 3.5)	Generación de planes de acción personalizados vía IA (estudiante)
Requests	Cliente HTTP para consumir la API externa
Git / GitHub	Control de versiones
Streamlit Cloud	Despliegue continuo

4. Modelo matemático (v5)

El modelo se ha calibrado con datos de la **ENUT 2024-2025** del DANE y se apoya en literatura complementaria (EAFIT 2010, UdeG 2013, Guevara 2021). Su función es simular el impacto de las cargas no académicas sobre el rendimiento, con fines de reflexión; varios coeficientes se basan en supuestos informados. **Es el mismo modelo que se aplica tanto al estudiante individual como al procesamiento masivo del rol administrativo**.

4.1. Variables de entrada

Variable	Tipo	Rango / Opciones	Notas
Género	Categórica	Femenino / Masculino	Ajuste de parámetros
Estrato	Numérico entero	1–6	Factor de ponderación
Composición del hogar	Categórica	Vive solo/a, con familia, con pareja, residencia universitaria, otros	Modula apoyo
Dependientes	Numérico entero	0, 1, 2, 3+	Personas a cargo
Horas domésticas	Numérico (float)	0–168	Desglose opcional
Trabajo remunerado	Sí/No + horas	0–168	
Horas de estudio	Numérico	0–168	
Promedio actual (opcional)	Numérico	0–5.0	Para comparación
Solo para el rol administrativo (además de las anteriores):			
id_estudiante	Cadena	Identificador único anonimizado	No se almacena información personal

4.2. Carga total y fatiga cognitiva

La **carga total** es la suma del trabajo doméstico y laboral (las horas de estudio no se incluyen, pues se consideran la variable sobre la que impacta la fatiga):

$$\text{carga_total} = \text{horas_domésticas} + \text{horas_trabajo}.$$

La fatiga cognitiva se modela con una función sigmoide cuyo punto de inflexión y pendiente se han fijado para la población estudiantil colombiana:

$$\mu = 45 \text{ horas/semana}, \quad \kappa = 10.$$

Los factores de ajuste $\alpha_{\text{género}}$, β_{estrato} y γ_{hogar} modulan la pendiente de la sigmoide, no su salida, para evitar saturaciones artificiales:

$$\text{fatiga} = \sigma\left(\frac{\text{carga_total} - \mu}{\kappa/(\alpha \cdot \beta \cdot \gamma)}\right), \qquad \sigma(x) = \frac{1}{1 + e^{-x}}.$$

El producto $\alpha\beta\gamma$ se limita a un valor máximo de 1.40.

4.2.1. Factor alfa - Género

La ENUT 2024-2025 reporta que las mujeres dedican en promedio 7 horas y 35 minutos diarios a trabajo no remunerado, frente a 3 horas y 12 minutos de los hombres. La razón resultante es $7{,}58/3{,}20 \approx 2{,}37$. Para no duplicar el efecto de la carga horaria ya declarada, se toma la raíz cuadrada ($\sqrt{2{,}37} \approx 1{,}54$) y se redondea a 1,40 (el techo del factor combinado):

$$\alpha_{\text{Femenino}} = 1{,}40, \quad \alpha_{\text{Masculino}} = 1{,}00.$$

4.2.2. Factor beta - Estrato

Basado en el acceso diferencial a servicios de externalización del hogar (ECV 2023). Los estratos bajos carecen de dichos servicios, lo que incrementa la carga real.

Estrato	β
1	1,25
2	1,18
3	1,08
4	1,00
5	0,93
6	0,87

4.2.3. Factor gamma - Composición del hogar

Ajustado según la distribución de carga doméstica por tipo de hogar (ENUT 2024-2025, gráficos 9-10). Los hogares extensos presentan mayor demanda; la residencia universitaria

reduce la carga por servicios incluidos.

Hogar	γ
Residencia universitaria	0,75
Vive solo/a	0,90
Con pareja	0,95
Con familia	1,05
Otros	1,00

4.3. Horas de estudio efectivas

$$\text{horas_efectivas} = \text{horas_estudio} \times (1 - \text{fatiga}).$$

La fatiga actúa como un descuento proporcional: con fatiga $= 0.6$ solo el 40 % del tiempo de estudio se traduce en aprendizaje de calidad.

4.4. Estimación del promedio (PPA)

Se emplea un modelo lineal con interacción género×fatiga:

$$\begin{aligned}\text{PPA_estimado} = \beta_0 &+ \beta_1 \cdot \text{horas_efectivas} + \beta_2 \cdot \text{estrato} \\ &+ \beta_3 \cdot \mathbb{1}_{\text{Femenino}} + \beta_4 \cdot \text{dependientes} \\ &+ \beta_5 \cdot (\mathbb{1}_{\text{Femenino}} \times \text{fatiga}).\end{aligned}$$

Los coeficientes son supuestos basados en la literatura:

Coef.	Valor	Fuente / Justificación
β_0	3,00	Intercepto modal del sistema (3.0/5.0)
β_1	0,025	UdeG 2013, Fazio 2004
β_2	0,10	EAFIT 2010
β_3	−0,10	Penalización residual de género
β_4	−0,07	Efecto marginal por dependiente
β_5	−0,35	Interacción género×fatiga

El efecto máximo combinado para una mujer con fatiga extrema es −0,45 puntos, un valor diseñado para la sensibilización sin caer en determinismos excesivos.

4.5. Shadow-Score

Para mejorar su interpretabilidad, el índice se descompone en dos componentes ponderados al 50 %:

$$\begin{aligned}\text{comp_tiempo} &= \frac{\text{carga_total}}{168}, \\ \text{comp_fatiga} &= \text{fatiga}, \\ \text{Shadow-Score} &= 100 \times \big(0{,}5 \cdot \text{comp_tiempo} + 0{,}5 \cdot \text{comp_fatiga}\big).\end{aligned}$$

Categorías cualitativas: 0–20 baja penalización, 21–40 moderada, 41–60 significativa, 61–80 alta, 81–100 extrema.

4.6. Coste de oportunidad

$$\text{coste_horas} = \text{horas_estudio} - \text{horas_efectivas},$$
$$\text{coste_ppa} = \text{PPA_actual} - \text{PPA_estimado} \quad \text{(si se ingresó el promedio actual).}$$

5. Asistente IA: Planes de acción personalizados

La aplicación incorpora un módulo de inteligencia artificial que, a partir de los datos del usuario y del coste de oportunidad, genera recomendaciones concretas. Este módulo está disponible exclusivamente para el rol estudiante; no se activa en el módulo administrativo para proteger la confidencialidad de los datos sensibles.

5.1. Lógica del asistente

Se utiliza la **NVIDIA API** con el modelo `mistralai/mistral-medium-3.5-128b` mediante un prompt estructurado que incluye:

- Perfil del usuario (género, estrato, composición del hogar, dependientes).
- Carga detectada (horas domésticas, laborales, fatiga cognitiva).
- Coste de oportunidad (diferencia de horas y pérdida de promedio).
- Contexto del escenario (Shadow-Score y métricas actuales).

5.2. Funcionalidad

1. Botón "Generar plan de acción con IA"junto a los resultados.
2. Llamada a la API con clave gestionada mediante `st.secrets` (Streamlit Cloud) o `.streamlit/secrets.toml` (local).
3. Visualización de sugerencias formateadas (estrategias de redistribución, apoyos institucionales, gestión del tiempo, recursos externos).
4. Incluye un aviso informativo (disclaimer).

5.3. Integración técnica

Componente	**Detalle**
API Externa	NVIDIA API (modelo `mistralai/mistral-medium-3.5-128b`)
Manejo de clave	`st.secrets` en Streamlit Cloud / `.streamlit/secrets.toml` en local
Prompt engineering	Archivo `IA_prompts.py` con placeholders
Llamada HTTP	`requests` con timeout extendido (10 min) y reintentos automáticos
Caché	Evita llamadas repetidas con el mismo perfil

6. Módulo administrativo: procesamiento masivo y dashboards

Este módulo está destinado a personal de la universidad (coordinadores, bienestar universitario, etc.) que necesita analizar la situación de un conjunto de estudiantes sin acceder a datos personales identificables y sin exponer información sensible a APIs externas. A continuación se describen su flujo, formato de entrada, proceso ETL (sin base de datos, usando estructuras internas de Python) y los dashboards de presentación.

6.1. Formato del archivo CSV de entrada

El archivo debe contener una fila por estudiante. Las columnas obligatorias son las mismas que se definieron en la tabla de variables de entrada (sección 4.1), más una columna adicional:

- `id_estudiante`: identificador anónimo (p.ej., un hash del código estudiantil). No se requiere ninguna otra columna de segmentación (como facultad o programa), ya que la universidad cargará un archivo por cada facultad o unidad académica.

El archivo puede incluir opcionalmente la columna `promedio_actual` para calcular el coste de oportunidad en términos de puntos de calificación.

6.2. Proceso ETL (Extracción, Transformación, Carga)

El flujo se implementa íntegramente en Python utilizando Pandas y estructuras internas (DataFrames), sin necesidad de una base de datos externa. Se ejecuta dentro de la aplicación Streamlit bajo autenticación previa del usuario administrativo. Las etapas son:

1. **Extracción**: el usuario sube el archivo CSV mediante un widget de Streamlit. Se lee con `pd.read_csv()` y se verifica la codificación (UTF-8 o Latin-1). El contenido se almacena temporalmente en un DataFrame.

2. **Validación**:

   - Comprobar la existencia de todas las columnas obligatorias (incluyendo `id_estudiante`).
   - Verificar rangos: horas entre 0 y 168, estrato entre 1 y 6, dependientes en {0,1,2,3+} (se interpreta 3+ como 3), etc.
   - Consistencia: si el archivo incluye una columna `trabaja` (Sí/No), validar que `horas_trabajo` sea 0 cuando no trabaja.

   Las filas con errores críticos se descartan y se registran en un log que se muestra al usuario.

3. **Limpieza**:

   - Eliminación de filas duplicadas por `id_estudiante` (se conserva la primera).
   - Imputación simple de valores faltantes en variables no críticas (por ejemplo, si falta `promedio_actual`, se omite el cálculo de coste_ppa).

- Anonimización: se garantiza que ningún dato de identificación personal (nombres, documentos, correos) esté presente. Si el archivo los contuviera, se excluyen automáticamente.

4. **Transformación**:
   - Por cada fila del DataFrame, se aplica el modelo matemático (secciones 4.2 a 4.6) y se calculan:
     - Fatiga cognitiva
     - Horas efectivas de estudio
     - Shadow-Score (numérico y categoría cualitativa)
     - Coste de oportunidad en horas
     - Coste de oportunidad en PPA (si se suministró promedio_actual)
   - Se añaden estas columnas derivadas al DataFrame.
5. **Carga y almacenamiento interno**: Los resultados (datos originales anonimizados más columnas derivadas) se mantienen en memoria como un DataFrame de Pandas durante la sesión del usuario administrativo. No se utiliza ninguna base de datos externa; toda la información se gestiona con estructuras internas de Python. El usuario puede exportar los resultados agregados (no individuales) en formatos CSV o Excel si lo desea.

6.3. Dashboards de presentación de resultados

Una vez completado el procesamiento, el sistema muestra un conjunto de **dashboards interactivos** construidos con Plotly y Streamlit. Los dashboards se actualizan dinámicamente según la muestra cargada. Se ha priorizado la claridad y la utilidad para la toma de decisiones institucionales, sin incluir IA en esta etapa. Los paneles incluyen:

- **Resumen general**: tarjetas con número total de estudiantes procesados, promedio de Shadow-Score, distribución porcentual por categorías cualitativas (baja, moderada, significativa, alta, extrema).
- **Distribución de fatiga y cargas**: histogramas y gráficos de densidad de Shadow-Score, fatiga cognitiva, horas efectivas de estudio.
- **Análisis de brechas de género**: comparación de medias de horas domésticas, fatiga, Shadow-Score y horas efectivas entre mujeres y hombres.
- **Segmentación por estrato y composición del hogar**: gráficos de barras agrupadas que muestran el Shadow-Score promedio en función del estrato socioeconómico y del tipo de hogar.
- **Simulación agregada de corresponsabilidad**: permite al administrativo aplicar una reducción porcentual (por defecto 40 %) a las horas domésticas de todos los estudiantes y visualizar cómo cambian los indicadores agregados (Shadow-Score medio, porcentaje en riesgo extremo, etc.) en comparación con el escenario actual.

- **Exportación de reportes**: opción para descargar los resultados agregados (no datos individuales) en formato CSV, Excel o PDF, para su uso en informes institucionales.

Todos los dashboards se actualizan automáticamente al cargar un nuevo archivo. Para preservar la confidencialidad, no se muestran identificadores de estudiantes en ninguna vista.

7. Flujo de usuario y funcionalidades

A continuación se describen los flujos diferenciados para cada rol.

7.1. Rol estudiante

1. **Página de inicio**: presentación, ODS 5, datos DANE.
2. **Formulario de entrada**: datos sociodemográficos, carga de trabajo, académicos y percepción.
3. **Cálculo y visualización**: Shadow-Score, fatiga (gauge), horas efectivas, promedio estimado, gráfico de radar, coste de oportunidad.
4. **Simulación**: escenario actual vs. mejorado.
5. **Asistente IA**: botón para generar plan de acción personalizado.
6. **Información adicional**: metodología, fuentes, recursos institucionales.

7.2. Rol administrativo

1. **Acceso autenticado** al panel administrativo.
2. **Carga de archivo CSV** con la estructura definida (incluyendo `id_estudiante`).
3. **Procesamiento automático (ETL)** con validación en tiempo real y notificación de errores, utilizando exclusivamente estructuras internas de Python (Pandas).
4. **Visualización de dashboards** con los indicadores agregados y la simulación de corresponsabilidad a nivel de muestra.
5. **Exportación de reportes** para su uso en documentos institucionales.

8. Alcances y límites del desarrollo actual

Esta sección describe de manera precisa qué funcionalidades están completamente implementadas en el código actual, cuáles están parcialmente desarrolladas y cuáles fueron planteadas en el diseño pero no se encuentran operativas en la versión desplegada.

8.1. Alcances implementados

8.1.1. Módulo estudiante

- Formulario completo de entrada de datos sociodemográficos, carga doméstica, laboral y académica, con validación de rangos en tiempo real.
- Ejecución del modelo matemático completo (fatiga, horas efectivas, PPA estimado, Shadow-Score y coste de oportunidad) sobre los datos ingresados.
- Visualización de resultados mediante gráficos interactivos con Plotly: indicador de fatiga tipo gauge, gráfico de radar multivariable y métricas de coste de oportunidad.
- Simulación de escenario mejorado: el usuario puede ajustar la redistribución de carga doméstica y visualizar el impacto en sus indicadores en comparación con el escenario actual.
- Integración funcional con la NVIDIA API (modelo `mistralai/mistral-medium-3.5-128b`) para generación de planes de acción personalizados, con timeout extendido (10 minutos) y reintentos automáticos ante errores 5xx.
- Gestión segura de la API key mediante `st.secrets` en Streamlit Cloud y `.streamlit/secrets.toml` en entorno local.

8.1.2. Módulo administrativo

- Carga de archivos CSV mediante widget de Streamlit con validación automática de columnas obligatorias, rangos de valores y consistencia entre campos.
- Flujo ETL completo implementado con Pandas: extracción, validación, limpieza (eliminación de duplicados por `IDEstudiante`, imputación de valores faltantes no críticos) y transformación aplicando el modelo matemático a cada registro.
- Generación de columnas derivadas: `fatiga`, `horas_efectivas`, `shadow_score` y `categoria` cualitativa.
- Dashboard interactivo con cuatro gráficos Plotly: distribución de Shadow-Score por categoría, fatiga promedio por estrato, brecha de género en Shadow-Score y distribución de horas efectivas de estudio.
- Tabla de resultados completa con todos los registros procesados, visible en la interfaz con formato condicional.
- Persistencia de resultados durante la sesión activa mediante `st.session_state`.

8.1.3. Infraestructura y despliegue

- Aplicación desplegada en Streamlit Cloud con URL pública permanente, integración continua desde el repositorio GitHub y gestión de secrets en la plataforma.
- Estructura modular del proyecto: separación clara entre `Back_end/` (lógica de negocio), `config/` (estilos, colores, constantes) y `pages/` (vistas Streamlit).
- Compatibilidad con Python 3.14 mediante versiones actualizadas de dependencias (`pandas>=2.2.0`, `numpy>=2.0.0`).

8.2. Límites del desarrollo actual

8.2.1. Ausencia de autenticación real

El módulo administrativo no implementa un sistema de autenticación. Cualquier usuario que acceda a la URL de la aplicación puede navegar al panel administrativo sin restricción de credenciales. El informe describe un "acceso autenticado" que no está operativo en el código actual.

8.2.2. Sin exportación de reportes

El dashboard administrativo muestra los resultados procesados en pantalla pero no ofrece funcionalidad de exportación a CSV, Excel ni PDF. Esta característica fue planteada en el diseño pero no fue implementada en la versión actual.

8.2.3. Sin persistencia entre sesiones

Los datos procesados (DataFrame de resultados) se almacenan exclusivamente en `st.session_state`, lo que implica que se pierden al cerrar o recargar la aplicación. No existe base de datos ni almacenamiento externo. Cada sesión administrativa debe reiniciar la carga del CSV desde cero.

8.2.4. Modelo con coeficientes supuestos

Varios coeficientes del modelo de estimación del PPA (β_0, β_3, β_4, β_5) están marcados explícitamente en el código como `[SUPUESTO]`, ya que no se derivaron de regresión sobre datos locales. El propio modelo advierte que deben reemplazarse con regresión cuando se cuente con $n \geq 150$ registros reales.

8.2.5. Género binario

El modelo matemático y el formulario del estudiante contemplan únicamente dos opciones de género (Femenino / Masculino), lo que limita la inclusión de estudiantes no binarios o de otras identidades de género. Esta restricción proviene del diseño actual de los factores de ajuste basados en la ENUT.

8.2.6. Simulación de corresponsabilidad solo a nivel individual

La simulación de escenarios de redistribución de carga doméstica está implementada únicamente para el rol estudiante (vista individual). El dashboard administrativo no incluye la simulación agregada de corresponsabilidad descrita en el diseño, que permitiría aplicar una reducción porcentual a todos los registros y comparar indicadores colectivos.

8.2.7. Sin caché de llamadas a la IA

Aunque el informe menciona caché para evitar llamadas repetidas con el mismo perfil, el código actual no implementa ningún mecanismo de caché (`st.cache_data` o similar) sobre las llamadas a la NVIDIA API. Cada solicitud genera una nueva llamada independientemente de si el perfil ya fue procesado anteriormente.

8.2.8. Columna `promedio_actual` sin integración en dashboards

El ETL admite la columna opcional `promedio_actual` para calcular el coste de oportunidad en PPA, pero el dashboard no visualiza este indicador agregado. Los gráficos actuales no incluyen distribución del coste_ppa ni comparativa entre PPA actual y estimado a nivel institucional.

9. Conclusión

La versión 5 de Shadow-Score Académico consolida un modelo matemático fundamentado en la ENUT 2024-2025 del DANE, que evita saturaciones y ofrece indicadores interpretables. El asistente de IA, implementado mediante la NVIDIA API con el modelo Mistral Medium 3.5, convierte la reflexión en planes de acción concretos, mientras que la arquitectura en Streamlit asegura una herramienta robusta, escalable y alineada con los objetivos de sensibilización y equidad de género. La incorporación del módulo administrativo, con procesamiento masivo de CSV mediante ETL implementado con estructuras internas de Python (Pandas) y dashboards institucionales, permite a la universidad realizar análisis agregados sin exponer datos sensibles a servicios externos, cumpliendo con las normativas de protección de datos y fortaleciendo la toma de decisiones en políticas de permanencia y equidad de género.